

Pemrograman Berorientasi Object

Inheritance

TIM PBO

Inheritance

- Pengertian Dasar Inheritance
 - Deklarasi Inheritance
 - Jenis Inheritance
 - Kapan menerapkan inheritance?
 - Pengaksesan member dari parent class
 - Kontrol pengaksesan
 - Kata kunci *super*
 - Konstruktor tidak diwariskan
- 

Pengertian Dasar Inheritance

- Konsep inheritance adalah mengadopsi dunia riil, dimana object dapat memiliki turunan.
- Dengan konsep inheritance, sebuah class dapat memiliki class turunan.
- Suatu class yang memiliki class turunan dinamakan **parent class**, atau **base class**, atau **super class**.
- Sedangkan class turunan disebut juga dengan **child class** atau **sub class**.
- Suatu sub class mewarisi semua class member yang ada pada super class nya (properties, behaviour, dsb) kecuali member private dan konstruktor.
- Kesimpulannya, suatu sub class adalah perluasan (extends) dari parent class nya.

Kapan Menggunakan Inheritance?

- Misalkan pada sebuah game, terdapat karakter yang memiliki atribut **Name**, **Level**, **Health** dan method **attack()**
- Di Dalam game tersebut terdapat beberapa jenis karakter, yaitu **Angel**, **Human**, dan **Wizard**. masing-masing karakter memiliki karakteristik yang sama, namun ada beberapa karakteristik yang membedakan:
 - **Angel** memiliki atribut **Potion** dan method **cure()**
 - **Human** memiliki atribut **Strength** dan method **specialAttack()**
 - **Wizard** memiliki atribut **Spell** dan method **magic()**
- Namun semua jenis karakter tersebut tetap memiliki atribut Name, Level dan Health.
- Amati class diagram berikut:

Kapan Menggunakan Inheritance?

Human
+ Name: String + Level: int + Health: int + Strength: int
+ attack() + specialAttack()

Angel
+ Name: String + Level: int + Health: int + Potion: int
+ attack() + cure()

Wizard
+ Name: String + Level: int + Health: int + Spell: int
+ attack() + magic()

Jika diamati, class-class diatas memiliki beberapa atribut dan method yang sama.

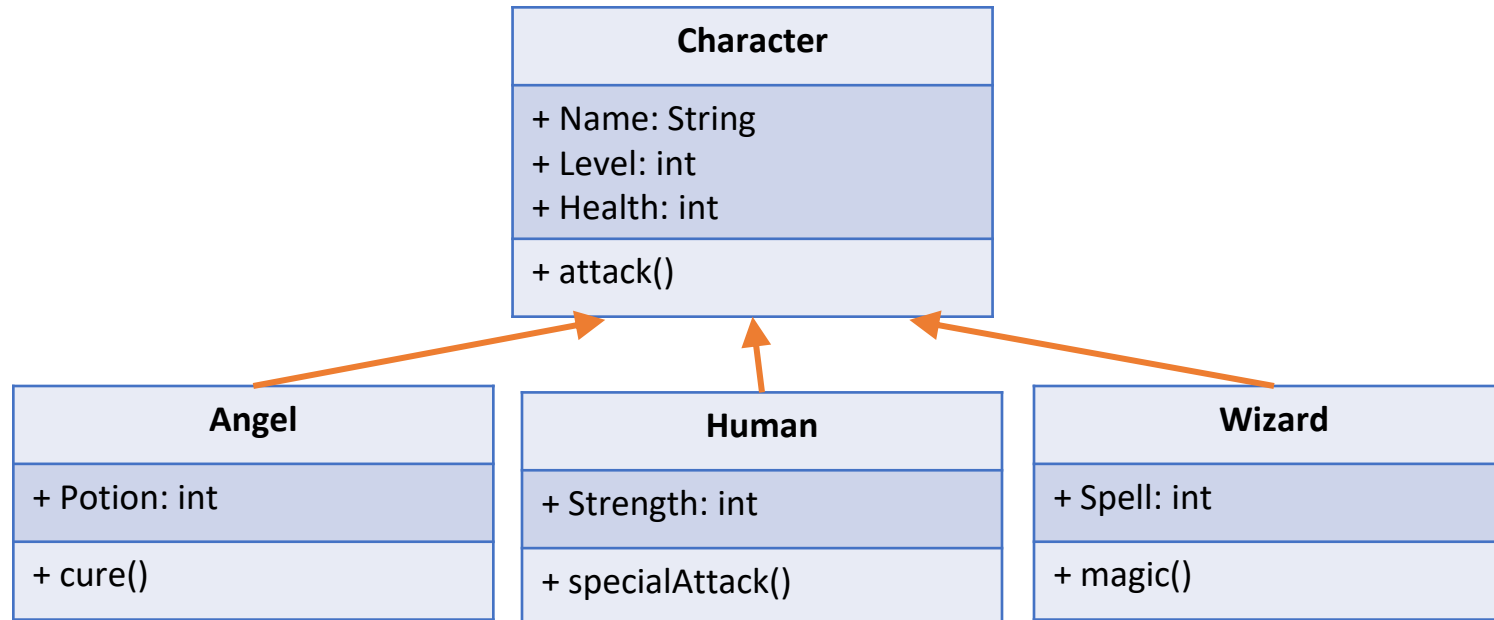
- Sehingga terjadi penulisan kode program yang berulang.

Solusinya?

- Dengan memanfaatkan inheritance, kita bisa membuat parent class Character yang memiliki atribut dan method tertentu yang bisa diturunkan ke class yang lain.
- Sehingga mengurangi pengulangan kode program.

Amati class diagram berikut:

Kapan Menggunakan Inheritance?



- Dengan menggunakan inheritance, kita buat *parent class* **Character** yang memiliki atribut Name, Level, Health dan method `attack()`
- Atribut dan method dari class Character tersebut diwariskan ke *child class* yaitu Angel, Human dan Wizard.

Deklarasi Inheritance

- Untuk mendeklarasikan suatu class menjadi sub class, dilakukan dengan cara menambahkan kata kunci **extends** setelah deklarasi nama class, kemudian diikuti nama parent class nya.

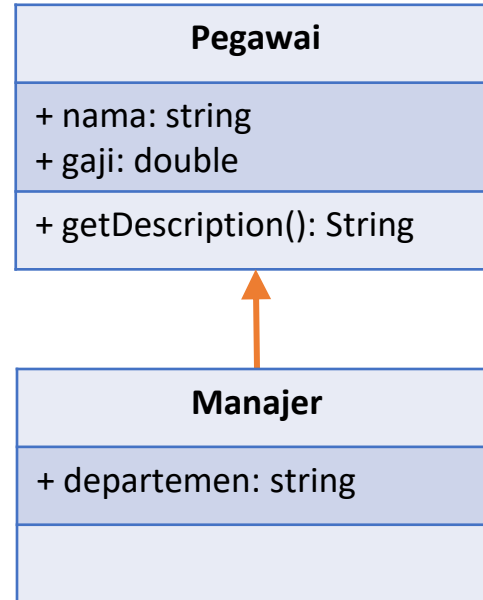
```
public class B extends A
{
    ...
}
```

- Contoh diatas memberitahukan compiler Java bahwa class **B** adalah turunan dari class **A**.

Single Inheritance

- Java hanya memperbolehkan adanya **single inheritance**.
 - Artinya, suatu subclass hanya dapat memiliki satu parent class. Namun suatu parent class dapat memiliki banyak sub class.
- 

Contoh 1



- Pada saat class Manajer menurunkan class Pegawai, maka ia mewarisi semua *class member* yang dimiliki oleh class Pegawai.
- Dengan demikian, class Manajer mempunyai semua class member yang diwarisi oleh Pegawai (**nama, gaji, getDescription**), ditambah dengan class member yang ia punyai (**departemen**).

Contoh 1 (lanjutan)

```
public class Pegawai
{
    public String nama;
    public double gaji;

    public String getDescription()
    {
        return "Nama: " + nama + ", gaji: " + gaji;
    }
}
```

```
public class Manajer extends Pegawai
{
    public String departemen;
}
```

```
public class TestPegawai
{
    public static void main(String[] args)
    {
        Manajer man = new Manajer();

        man.nama = "Bob";
        man.gaji = 9999999;
        man.departemen = "IT";

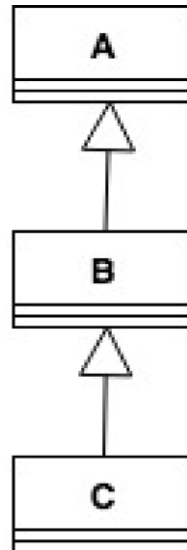
        System.out.println("Nama: " + man.nama);
        System.out.println("Gaji: " + man.gaji);
        System.out.println("Departemen: " + man.departemen);
    }
}
```

```
E:\Java>java TestPegawai
Nama: Bob
Gaji: 9999999.0
Departemen: Bob
```

Objek **man** dari class **Manajer** mengakses atribut **nama** dan **gaji** yang merupakan warisan dari class **Pegawai**

Multilevel Inheritance

- Suatu subclass bisa menjadi superclass bagi class yang lain



- Dapat dilihat bahwa B merupakan subclass dari A, dan C adalah subclass dari B

Kata Kunci *this* dan *super* (1)

- Kata kunci ***this*** digunakan untuk merujuk pada member dari class itu sendiri.
- Sedangkan kata kunci ***super*** merujuk pada member dari super class.
- Contoh kata kunci *this*

```
public class Siswa
{
    private int nrp;

    public void setNrp(int nrp)
    {
        this.nrp = nrp;
    }
}
```

Kata Kunci *this* dan *super* (2)

- Contoh kata kunci *super*

```
public class Parent
{
    public int x = 5;
}

public class Child extends Parent
{
    public int x = 10;

    public void Info(int x)
    {
        System.out.println("Nilai x sebagai parameter = " + x);
        System.out.println("Property x di class Child = " + this.x);
        System.out.println("Property x di class Parent = " + super.x);
    }
}
```

Kata Kunci *this* dan *super* (3)

```
public class TestProgram
{
    public static void main(String args[])
    {
        Child tes = new Child();
        tes.Info(20);
    }
}
```

- Hasil eksekusi:

```
D:\Java\Inheritance>java TestProgram
Nilai x sebagai parameter = 20
Property x di class Child = 10
Property x di class Parent = 5
```

Kontrol Pengaksesan (1)

- Member yang dideklarasikan sebagai ***private*** tidak dapat diwariskan.
- Amati contoh dibawah ini:

```
public class Pegawai
{
    private String nama;
    public double gaji;
}

public class Manajer extends Pegawai
{
    public String departemen;

    public void IsiData(String n, String d)
    {
        nama = n;
        departemen = d;
    }
}
```

Kontrol Pengaksesan (2)

- Ketika class **Manajer** diatas di-compile, maka akan terjadi error:

```
D:\Java\Inheritance>javac Manajer.java
Manajer.java:8: nama has private access in Pegawai
        nama = n;
        ^
1 error
```

- Hal ini membuktikan bahwa property **nama** pada class **Pegawai** tidak diwariskan kepada class **Manajer**, karena dideklarasikan sebagai ***private***.

Kontrol Pengaksesan (3)

- Pengaruh access modifier terhadap pengaksesan member dari class.

	Same Class Same Package	Other Class Same Package	SubClass Other Package	Other Class Other Package
public	✓	✓	✓	✓
protected	✓	✓	✓	
default	✓	✓		
private	✓			

Kontrol Pengaksesan (4)

- Coba lakukan perubahan pada class **Pegawai**, ubah access modifier dari property **nama** menjadi *protected* kemudian lakukan compile ulang pada class **Manajer** dan lihat apa yang terjadi.
- Seharusnya sudah tidak ada error, karena akses dari property **nama** sekarang sudah diubah jadi *protected*.

Konstruktor tidak diwariskan (1)

- Konstruktor tidak dapat diwariskan kepada sub class.
 - Namun sub class dapat memanggil konstruktor parent class dengan kata kunci **super()**.
 - Pemanggilan konstruktor parent class harus dilakukan pada konstruktor sub class di baris paling atas
- 

Konstruktor tidak diwariskan (2)

- Contoh:

```
public class Parent
{
    public Parent()
    {
        System.out.println("Ini adalah konstruktor Parent");
    }
}

public class Child extends Parent
{
    int x;

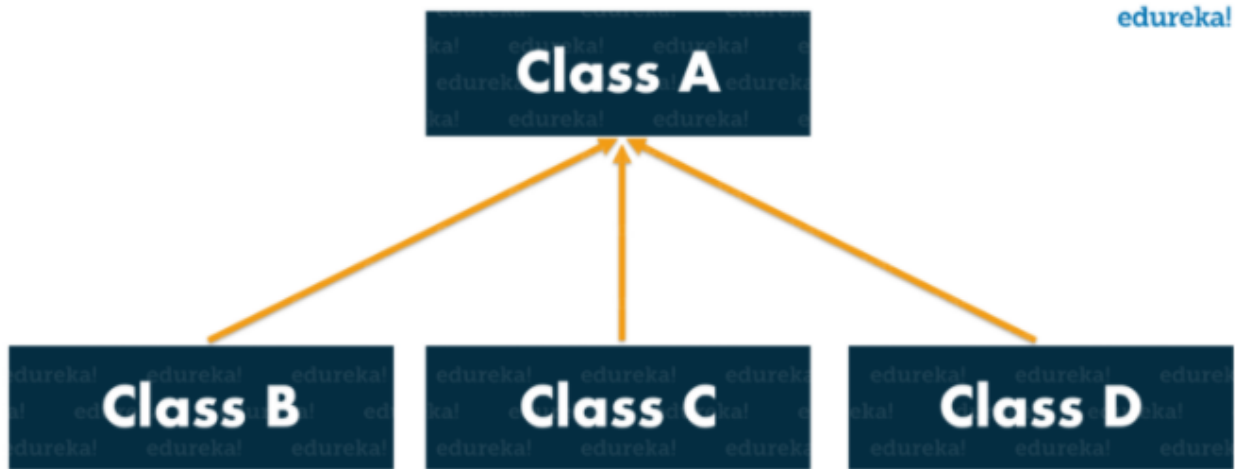
    public Child()
    {
        super();
        x = 5;
    }
}
```

Konstruktor class Child

Pemanggilan konstruktor Parent dilakukan di baris pertama didalam konstruktor Child

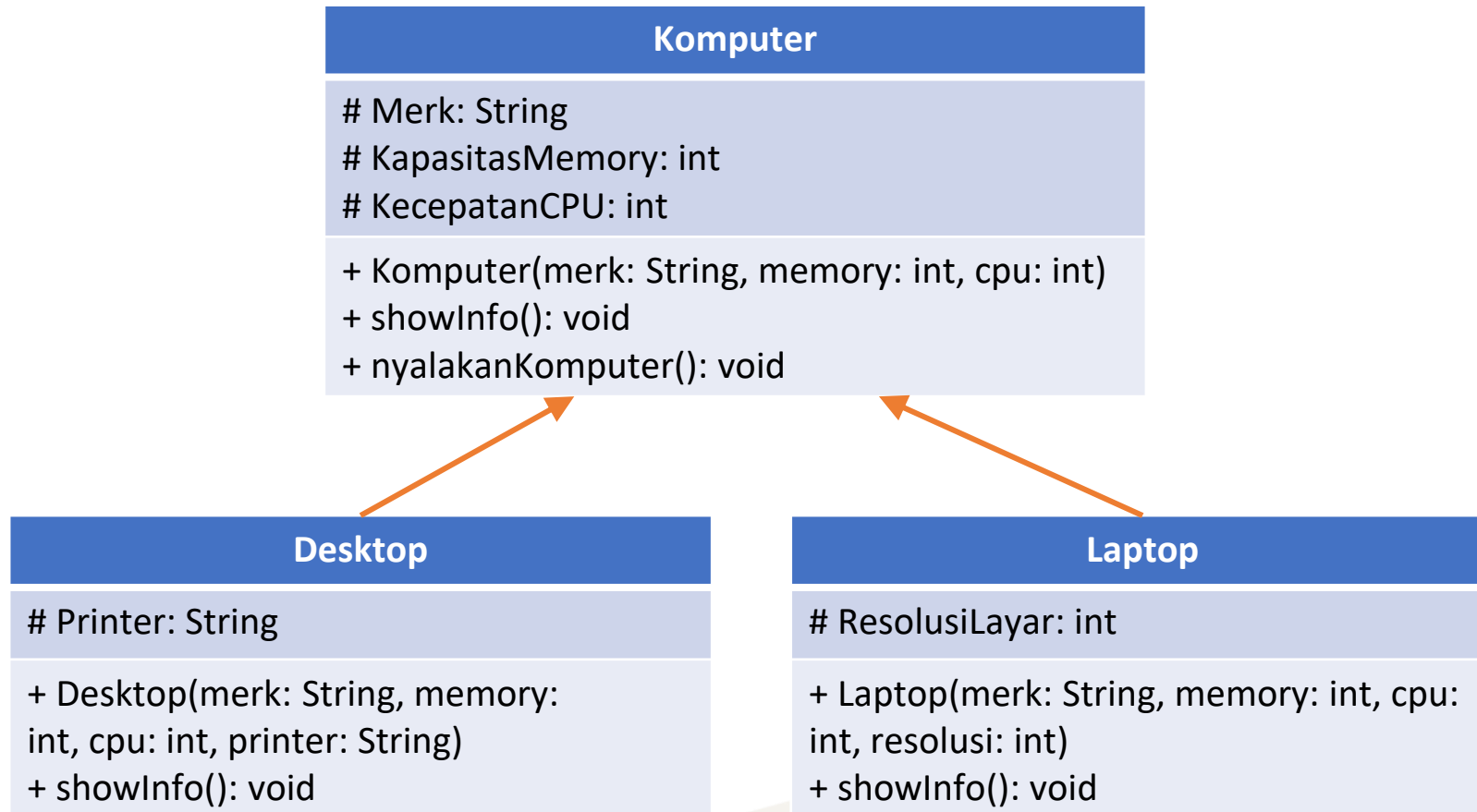
Hierarchical Inheritance

Ketika sebuah kelas memiliki lebih dari satu kelas turunan (subclass) atau dengan kata lain, lebih dari satu kelas turunan memiliki kelas induk yang sama.



```
class Animal{
    void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
    void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
    void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
    public static void main(String args[]){
        Cat c=new Cat();
        c.meow();
        c.eat();
        //c.bark();//C.T.Error
    }
}
```

Contoh 2



Contoh 2 (lanjutan)

- Uji dengan TestKomputer berikut ini:

```
public class TestKomputer
{
    public static void main(String[] args)
    {
        Desktop desk = new Desktop("Dell", 2048, 3500, "Canon");
        Laptop lap = new Laptop("Asus", 4096, 2500, 720);

        desk.showInfo();
        lap.showInfo();

        desk.nyalakanKomputer();
    }
}
```

Merk: Dell
Kapasitas Memory: 2048
Kecepatan CPU: 3500
Printer: Canon

Merk: Asus
Kapasitas Memory: 4096
Kecepatan CPU: 2500
Resolusi Layar: 720

Komputer Dell dinyalakan!

Contoh 2 (lanjutan)

```
public class Komputer
{
    protected String Merk;
    protected int KapasitasMemory;
    protected int KecepatanCPU;

    public Komputer(String merk, int memory, int cpu)
    {
        Merk = merk;
        KapasitasMemory = memory;
        KecepatanCPU = cpu;
    }

    public void showInfo()
    {
        System.out.println("\nMerk: " + Merk);
        System.out.println("Kapasitas Memory: " + KapasitasMemory);
        System.out.println("Kecepatan CPU: " + KecepatanCPU);
    }

    public void nyalakanKomputer()
    {
        System.out.println("\nKomputer " + Merk + " dinyalakan!");
    }
}
```

```
public class Desktop extends Komputer
{
    protected String Printer;

    public Desktop(String merk, int memory, int cpu, String printer)
    {
        super(merk, memory, cpu);
        Printer = printer;
    }

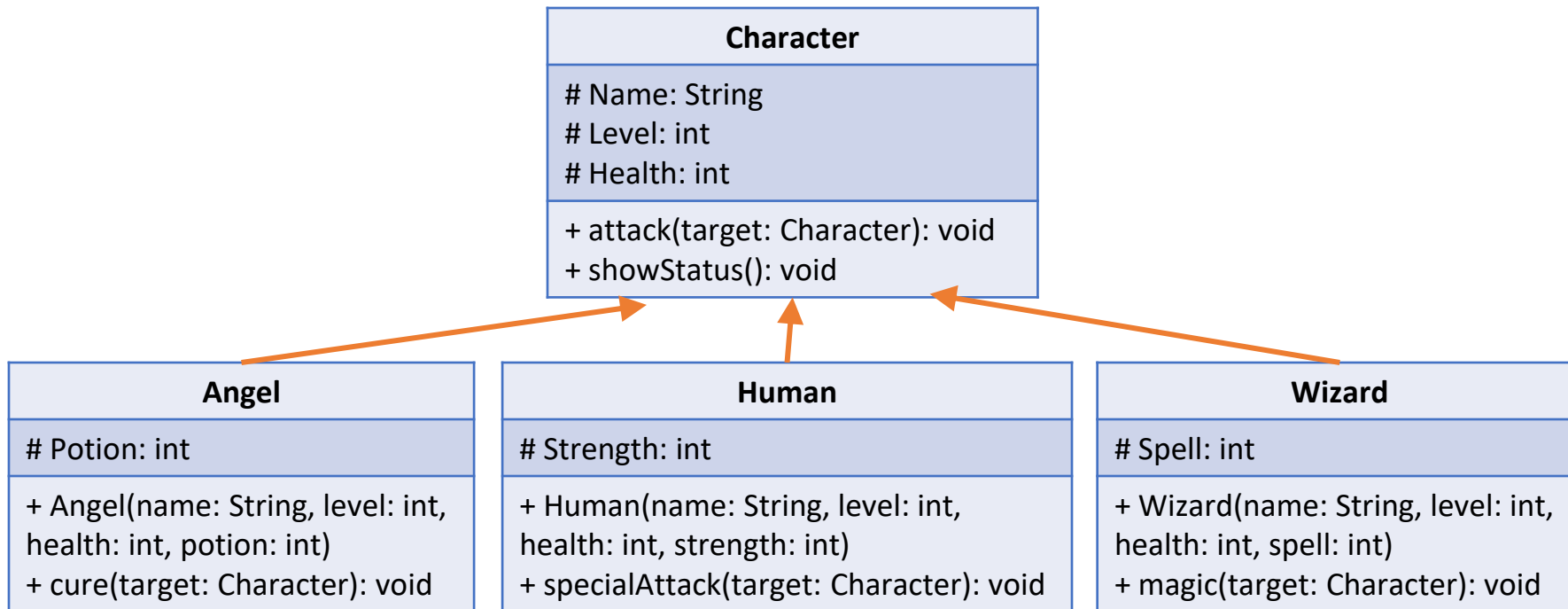
    public void showInfo()
    {
        super.showInfo();
        System.out.println("Printer: " + Printer);
    }
}
```

```
public class Laptop extends Komputer
{
    protected int ResolusiLayar;

    public Laptop(String merk, int memory, int cpu, int layar)
    {
        super(merk, memory, cpu);
        ResolusiLayar = layar;
    }

    public void showInfo()
    {
        super.showInfo();
        System.out.println("Resolusi Layar: " + ResolusiLayar);
    }
}
```

Contoh 3



- Semua karakter memiliki method `attack()` yang mengurangi health sebanyak 10.
- Angel memiliki method `cure()` yang berfungsi menyembuhkan, yaitu mengembalikan Health ke 100. Efeknya adalah mengurangi potion.
- Human memiliki method `specialAttack()` yang berfungsi mengurangi Health karakter sejumlah 10 + Strength yang dia miliki. Tidak ber efek mengurangi Strength.
- Wizard memiliki method `magic()` yang berfungsi mengurangi Health karakter sebanyak 50. Efeknya adalah mengurangi Spell.

Contoh 3 (lanjutan)

- Uji dengan test program:

```
public class TestAdventureGame
{
    public static void main(String[] args)
    {
        Angel esther = new Angel("Esther", 10, 100, 5);
        Human jackal = new Human("Jackal", 13, 100, 7);
        Wizard quistis = new Wizard("Quistis", 20, 100, 3);

        System.out.println("Begin game...");

        esther.showStatus();
        jackal.showStatus();
        quistis.showStatus();

        System.out.println("Jackal special attack to quistis, "
            + "quistis cast magic to jackal, "
            + "\nesther cure jackal, "
            + "quistis attack esther...");

        jackal.specialAttack(quistis);
        quistis.magic(jackal);
        esther.cure(jackal);
        quistis.attack(esther);

        esther.showStatus();
        jackal.showStatus();
        quistis.showStatus();
    }
}
```

Output yang diharapkan:

```
Begin game...
Name: Esther, Health: 100
Name: Jackal, Health: 100
Name: Quistis, Health: 100
Jackal special attack to quistis, quistis cast magic to jackal,
esther cure jackal, quistis attack esther...
Name: Esther, Health: 90
Name: Jackal, Health: 100
Name: Quistis, Health: 83
```

Contoh 3 (lanjutan)

```
public class Character
{
    protected String Name;
    protected int Level;
    protected int Health;

    public void attack(Character target)
    {
        target.Health -= 10;
    }

    public void showStatus()
    {
        System.out.println("Name: " + Name + ", Health: " + Health);
    }
}
```

```
public class Human extends Character
{
    protected int Strength;

    public Human(String name, int level, int health, int strength)
    {
        Name = name;
        Level = level;
        Health = health;
        Strength = strength;
    }

    public void specialAttack(Character target)
    {
        target.Health = target.Health - 10 - Strength;
    }
}
```

```
public class Angel extends Character
{
    protected int Potion;

    public Angel(String name, int level, int health, int potion)
    {
        Name = name;
        Level = level;
        Health = health;
        Potion = potion;
    }

    public void cure(Character target)
    {
        target.Health = 100;
        Potion -= 1;
    }
}
```

```
public class Wizard extends Character
{
    protected int Spell;

    public Wizard(String name, int level, int health, int spell)
    {
        Name = name;
        Level = level;
        Health = health;
        Spell = spell;
    }

    public void magic(Character target)
    {
        target.Health -= 50;
        Spell -= 1;
    }
}
```

Contoh 4

- Terdapat class Rekening, memiliki property Saldo. Terdapat method getSaldo yang mengembalikan nilai saldo. Terdapat method simpanUang yang menambahkan nilai saldo dengan jumlah uang yang disimpan. Terdapat method ambilUang yang mengurangi saldo dengan jumlah uang yang diambil.
- Terdapat class Deposito yang merupakan subclass dari Rekening. Class ini memiliki properti BungaPerBulan dan Tenor (jangka waktu deposito dalam bulan).
- Konstruktor menerima parameter saldo awal dan tenor, dimana saldo awal menjadi nilai property Saldo pada parent class (Rekening). Konstruktor juga mengeset nilai default BungaPerBulan yaitu 5%. Class ini memiliki method getNilaiBunga yang mengembalikan nilai deposito dengan rumus:

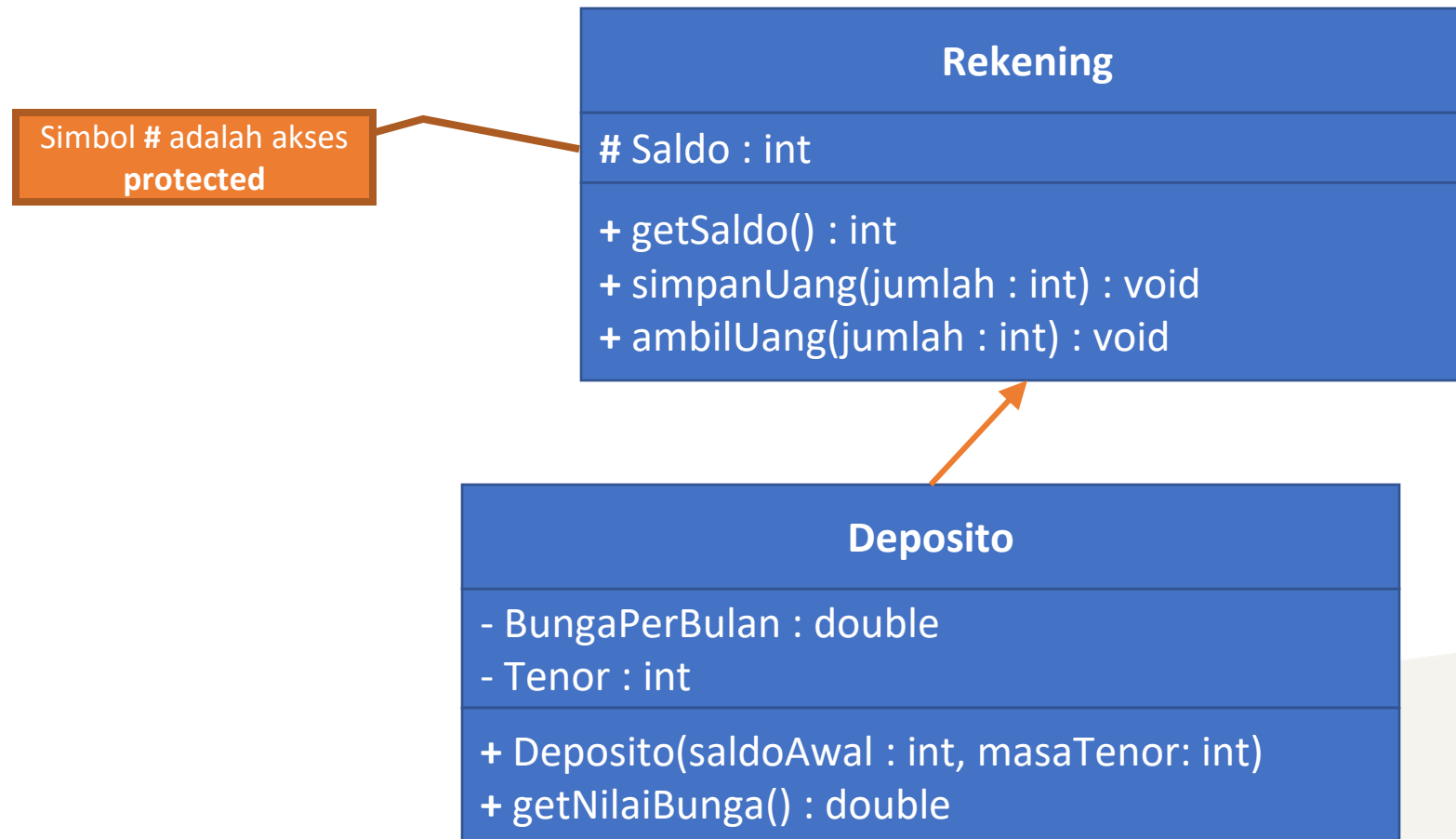
$$\text{NilaiBunga} = \text{Saldo} * \text{BungaPerBulan} * \text{Tenor} / 12$$

- Misal, saldo 1.000.000 dideposito dengan tenor 36 bulan dan bunga 1%, maka nilai bunga adalah:

$$\begin{aligned}\text{Nilai bunga} &= 1.000.000 * 1/100 * 36/12 \\ &= 30.000\end{aligned}$$

Contoh 4 (lanjutan)

- Class Diagram dari deskripsi diatas



Contoh 4 (lanjutan)

- Class Rekening

```
public class Rekening
{
    protected int Saldo;

    public int getSaldo()
    {
        return Saldo;
    }

    public void simpanUang(int jumlah)
    {
        Saldo = Saldo + jumlah;
    }

    public void ambilUang(int jumlah)
    {
        Saldo = Saldo - jumlah;
    }
}
```

saldo dibuat
protected agar dapat
diakses oleh class
turunannya

Contoh 4 (lanjutan)

○ Class Deposito

```
public class Deposito extends Rekening
{
    private double BungaPerBulan;
    private int Tenor;

    public Deposito(int saldoAwal, int masaTenor)
    {
        Saldo = saldoAwal;
        Tenor = masaTenor;
        BungaPerBulan = 0.05;
    }

    public double getNilaiBunga()
    {
        double nilaiBunga;

        nilaiBunga = Saldo * BungaPerBulan * Tenor / 12;

        return nilaiBunga;
    }
}
```

Mengakses property
saldo milik class
parent (Tabungan)

Contoh 4 (lanjutan)

- Class **TestDeposito** untuk pengetesan:

```
public class TestDeposito
{
    public static void main(String[] args)
    {
        Deposito dpt = new Deposito(1200000, 4);

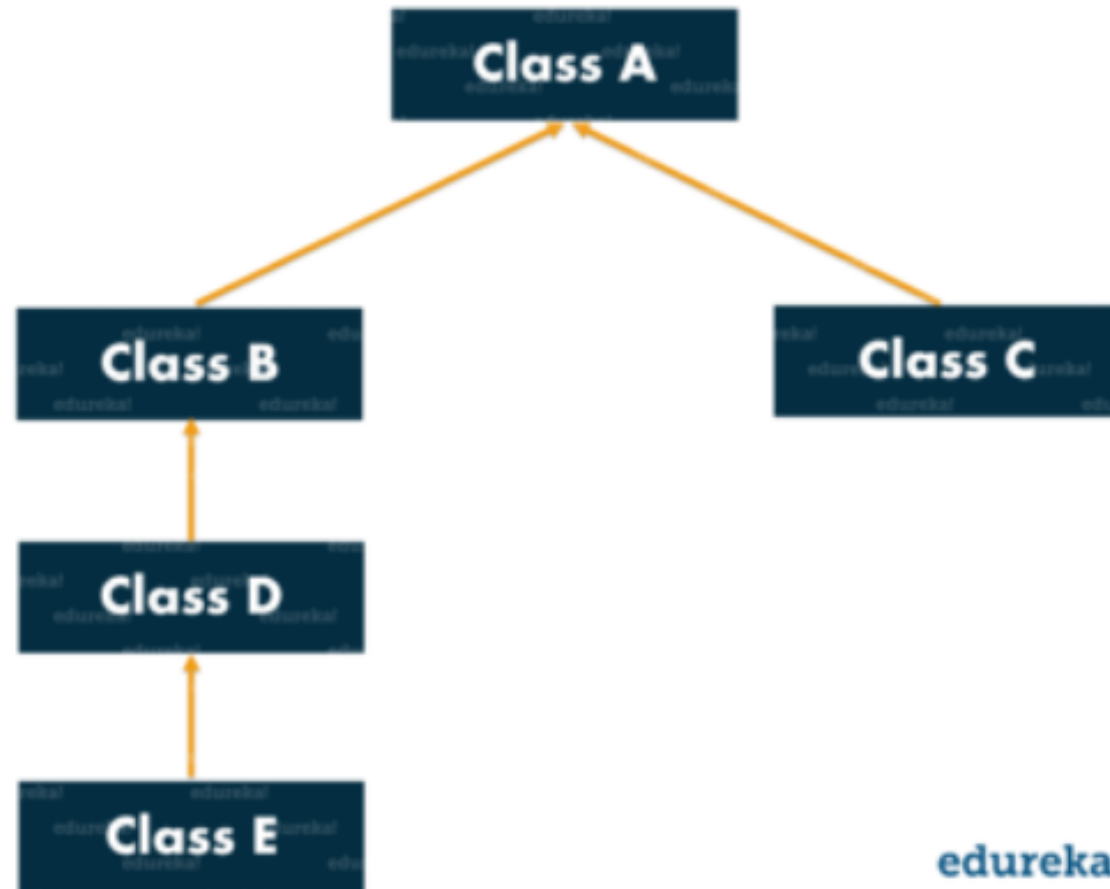
        System.out.println("Saldo awal: " + dpt.getSaldo());
        System.out.println("Nilai bunga deposito: " + dpt.getNilaiBunga());
        System.out.println("Total deposito: " + (dpt.getSaldo() + dpt.getNilaiBunga()));
    }
}
```

- Output:

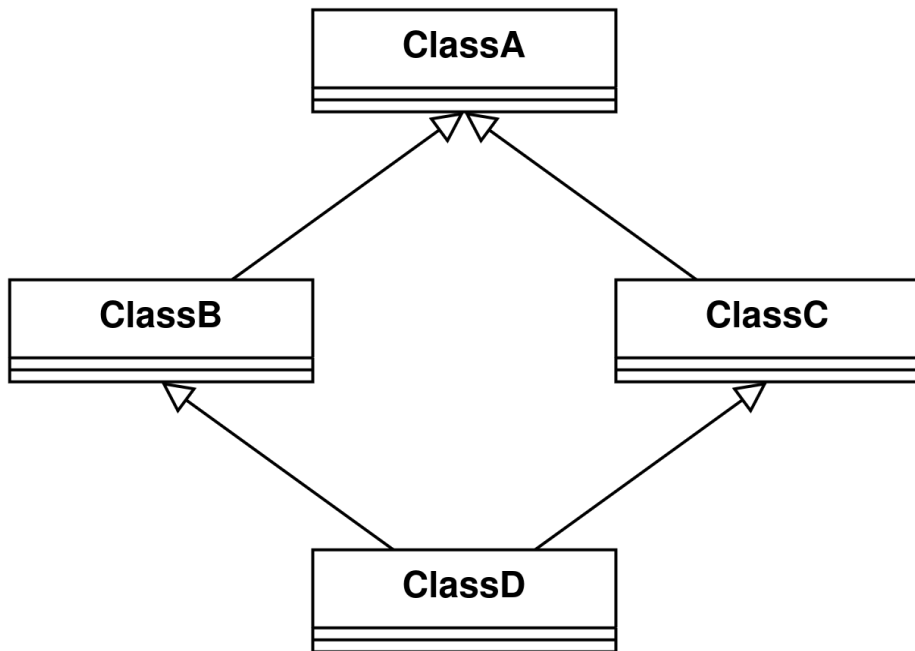
```
D:\Java>java TestDeposito
Saldo awal: 1200000
Nilai bunga deposito: 20000.0
Total deposito: 1220000.0
```

Hybrid Inheritance

Kombinasi dua atau lebih jenis inheritance.



Mengapa Multiple Inheritance tidak bisa digunakan di Java?



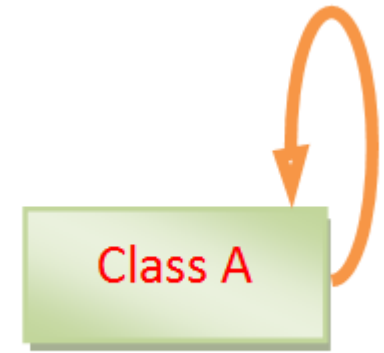
- Multiple inheritance mengacu pada proses dimana satu kelas turunan mencoba untuk memperluas (extend) lebih dari satu kelas induk.
- Misal terdapat method `show()` pada kelas B dan C dengan fungsi yang berbeda. Kemudian kelas D meng-extend kelas B dan C. Ketika objek dari kelas D mencoba memanggil method `show()`, kompilator akan bingung method di kelas mana yang akan dieksekusi (dari kelas B atau C)
- Mengarah pada ambiguitas.

Mengapa Multiple Inheritance tidak bisa digunakan di Java?

```
class A{  
    void message(){System.out.println("Hello");}  
}  
class B extends A{  
    void show(){System.out.println("Welcome");}  
}  
class C extends A{  
    void show(){System.out.println("Good Morning");}  
}
```

```
class D extends B,C{  
    void print(){System.out.println("This is multiple inheritance");}  
}  
class TestInheritance4{  
    public static void main(String args[]){  
        D obj=new D();  
        obj.print();  
        obj.show();//method show() mana yang akan dipanggil  
    }  
}
```

Rules Inheritance di Java



- Cyclic inheritance tidak dapat digunakan di Java.
Cyclic inheritance merupakan jenis inheritance dimana sebuah kelas memperluas (extend) dirinya sendiri. Jenis ini tidak diizinkan oleh Java karena tidak ada peluang untuk memperluas kelas Object.
- Atribut dan method dengan access modifier Private tidak diwariskan.
- Constructor tidak dapat diwariskan. Jika ingin menggunakan constructor kelas induk, harus menggunakan perintah `super()` pada constructor kelas anak.

Exercise 1

- Terdapat class Televisi dan TelevisiModern



Exercise 1 (lanjutan)

- Buat class **Televisi** dan **TelevisiModern** sesuai class diagram diatas.
- Kemudian **TestTelevisi** untuk pengetesan:

```
public class TestTelevisi
{
    public static void main(String[] args)
    {
        TelevisiModern tv = new TelevisiModern("Samsung", 100);
        System.out.println("Channel aktif: " + tv.getChannelAktif());
        tv.pindahChannel(20);
        System.out.println("Channel aktif sekarang: " + tv.getChannelAktif());
        tv.gantiModusTampilan("HDMI");
        tv.mainkanDVD();
        tv.masukkanDVD("The Matrix");
        tv.mainkanDVD();
    }
}
```

```
D:\Java\Inheritance>java TestTelevisi
Channel aktif: 1
Channel aktif sekarang: 20
Sedang memainkan DVD: kosong
Sedang memainkan DVD: The Matrix
```